

05 – Javascript

Interacting with web pages

Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Introduction to Javascript
- 2) History
- 3) Javascript syntax
- 4) Javascript functions
- 5) Javascript objects
- 6) Javascript arrays
- 7) Interactions Javascript/DOM
- 8) Examples
- 9) Best practices
- 10) Resources about Javascript

1) INTRODUCTION TO JAVASCRIPT



- **Javascript (JS)** is a programming language and core technology of the web platform, alongside HTML and CSS. Ninety-nine percent of websites on the World Wide Web use JavaScript on the client side for webpage behavior.
- Web browsers have a dedicated **JavaScript engine** that executes the client code. These engines are also utilized in some servers and a variety of apps. The most popular runtime system for non-browser usage is Node.js.

1) INTRODUCTION TO JAVASCRIPT



- JavaScript is a **high-level**, often **just-in-time-compiled** language that conforms to the **ECMAScript standard**. It has **dynamic typing**, **prototype-based object-orientation**, and first-class functions. It is multi-paradigm, supporting **event-driven, functional, and imperative** programming styles.
- It has application programming interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the **Document Object Model (DOM)**.

2) HISTORY

- 1995: Created by **Brendan Eich** at Netscape in just 10 days, originally named "Mocha," then "LiveScript," and finally "JavaScript." Released with Netscape Navigator 2.0.
- 1997: JavaScript standardized as **ECMAScript** (ES1) by ECMA International to ensure cross-browser compatibility.
- 2005: Jesse James Garrett coined the term **AJAX** (Asynchronous JavaScript and XML), enabling dynamic web applications like Gmail and Google Maps.

2) HISTORY

- 2009: Release of **Node.js** by Ryan Dahl, allowing JavaScript to run on servers and revolutionizing backend development.
- 2015: ECMAScript 6 (ES6/ES2015) introduced major updates: **arrow functions**, classes, modules, and **promises**, modernizing JavaScript.
- 2016–2020: Annual ECMAScript updates (ES2016–ES2020) added features like **async/await**, optional chaining, and **dynamic imports**, improving developer experience.
- 2020s: JavaScript dominates web development, with **frameworks** like **React**, **Vue**, and **Angular**, and expands to mobile (React Native) and desktop (Electron) apps.

3) JAVASCRIPT SYNTAX

- Javascript is a programming language in which you can declare variables, functions, call functions, use mathematical operators...

```
JS syntax.js > ...
1  // This is an example file for Javascript syntax
2  let iterations=1000 // declares the number of iterations
3  const pi=3.14159265 // declares a constant
4
5  // declares a function
6  function calculatePi(numberOfIterations) {
7      //  $\pi^2/6 = \sum_{k=1}^{\infty} (1/k^2)$ 
8      let sum=0
9      for (k=1; k<=numberOfIterations; k++) {
10         sum += 1/k**2
11     }
12     // declares the use of the Math JS library
13     result = Math.sqrt(6*sum)
14     return result // returns the result
15 }
16
17 // calls the function and store the result
18 let myComputationOfPi=calculatePi(iterations)
19 console.log("Calculated value for pi: ", myComputationOfPi)
20
21 // computes the error between calculated pi and real value of it
22 let error=pi-myComputationOfPi
23 console.log(`Error for ${iterations} iterations: `, error)
24
```

3) JAVASCRIPT SYNTAX

- To run the code on the client-side, you can use an HTML page

<> syntax-example.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>My JS example</title>
7  </head>
8  <body>
9      <h1>Example of a JS code into an HTML page</h1>
10     <p>Here comes the result...</p>
11
12     <script src="syntax.js"></script>
13 </body>
14 </html>
15
```

JS syntax.js > ...

```
1  // This is an example file for Javascript syntax
2  let iterations=1000 // declares the number of iterations
3  const pi=3.14159265 // declares a constant
4
5  // declares a function
6  function calculatePi(numberOfIterations) {
7      // pi**2/6 = sum_{k=1}^{infinity}(1/k**2)
8      let sum=0
9      for (k=1; k<=numberOfIterations; k++) {
10         sum += 1/k**2
11     }
12     // declares the use of the Math JS library
13     result = Math.sqrt(6*sum)
14     return result // returns the result
15 }
16
17 // calls the function and store the result
18 let myComputationOfPi=calculatePi(iterations)
19 console.log("Calculated value for pi: ", myComputationOfPi)
20
21 // computes the error between calculated pi and real value of it
22 let error=pi-myComputationOfPi
23 console.log(`Error for ${iterations} iterations: `, error)
24
```


3) JAVASCRIPT SYNTAX

- To run the code on the client-side, you can use an HTML page

Example of a JS code into an HTML page

Here comes the result...

Where is the result?

```
<> syntax-example.html > ...
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
```

```
11
12 <script src="syntax.js"></script>
13 </body>
14 </html>
15
```

```
4 // declares the number of iterations
5 // declares a function
6 function calculatePi(iterations) {
7   // computes the value of pi using the JS library
8   let pi = 3.14159265 // declares a constant
9   // computes the value of pi using the JS library
10  let result = sqrt(6*sum)
11  return result // returns the result
12 }
13
14 // calls the function and store the result
15 let myComputationOfPi=calculatePi(iterations)
16 console.log("Calculated value for pi: ", myComputationOfPi)
17
18 // computes the error between calculated pi and real value of it
19 let error=pi-myComputationOfPi
20 console.log(`Error for ${iterations} iterations: `, error)
21
22
```

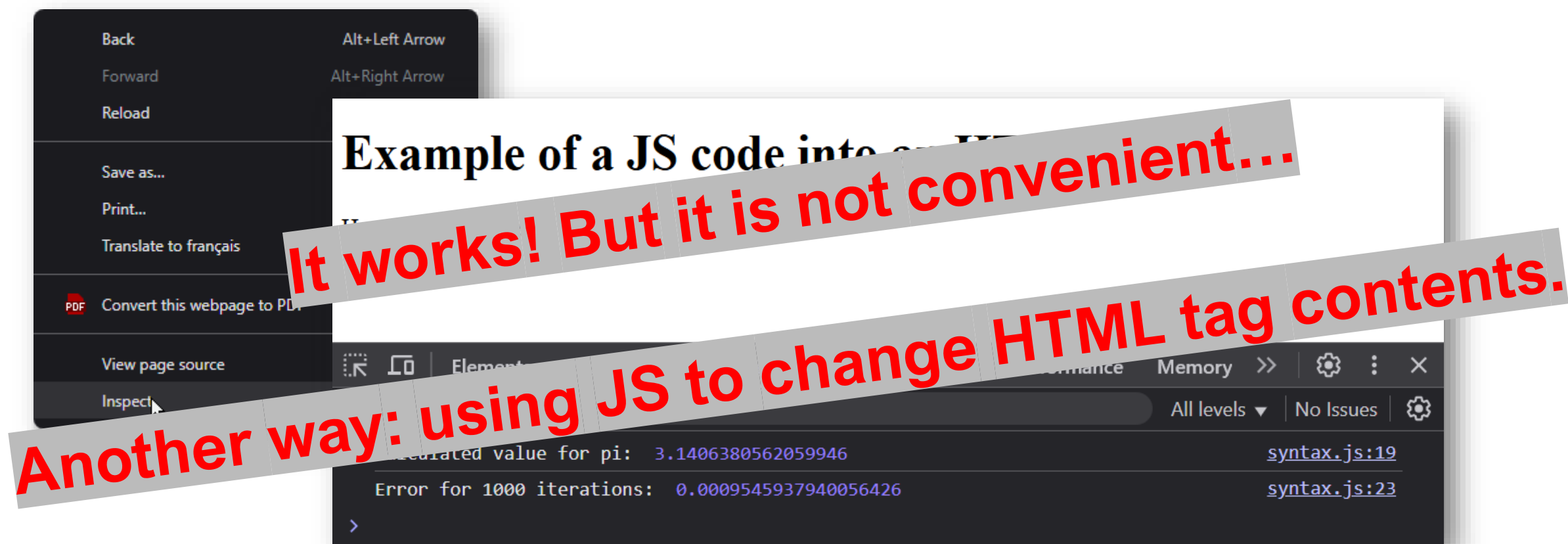
3) JAVASCRIPT SYNTAX

- We used the "console.log()" function to display the result, so it is in the console!



3) JAVASCRIPT SYNTAX

- We used the "console.log()" function to display the result, so it is in the console!



3) JAVASCRIPT SYNTAX

- We can use JS with interactions with the Document Object Model (DOM) to change HTML content live!

<> syntax-example.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, i
6    <title>My JS example</title>
7  </head>
8  <body>
9    <h1>Example of a JS code into an HTML page</h1>
10   <p id="result">Here comes the result...</p>
11   <p id="error">Here comes the error...</p>
12
13   <script src="syntax.js"></script>
14 </body>
15 </html>
```

JS syntax.js > ...

```
1  // This is an example file for Javascript syntax
2  let iterations=1000 // declares the number of iterations
3  const pi=3.14159265 // declares a constant
4
5  // declares a function
6  function calculatePi(numberOfIterations) {
7    // pi**2/6 = sum_{k=1}^{infinity}(1/k**2)
8    let sum=0
9    for (k=1; k<=numberOfIterations; k++) {
10      sum += 1/k**2
11    }
12    // declares the use of the Math JS library
13    result = Math.sqrt(6*sum)
14    return result // returns the result
15  }
16
17  // calls the function and store the result
18  let myComputationOfPi=calculatePi(iterations)
19  //console.log("Calculated value for pi: ", myComputationOfPi)
20  document.querySelector("#result").innerHTML=`Calculated value for pi: ${myComputationOfPi}`
21
22  // computes the error between calculated pi and real value of it
23  let error=pi-myComputationOfPi
24  //console.log(`Error for ${iterations} iterations: `, error)
25  document.querySelector("#error").innerHTML=`Error for ${iterations} iterations: ${error}`
26
```

3) JAVASCRIPT SYNTAX

- We can use JS with interactions with the Document Object Model (DOM) to change HTML content live!

<> syntax-example.html > ...

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1">
6    <title>My JS example</title>
7  </head>
8  <body>
9    <h1>Example of a JS code into an HTML page</h1>
10   <p id="result"></p>
11   <p id="error"></p>
12   <script src="syntax.js"></script>
13
14 </body>
15 </html>

```

Example of a JS code into an HTML page

Calculated value for pi: 3.1406380562059946

Error for 1000 iterations: 0.0009545937940056426

JS syntax.js > ...

```

1  // This is an example file for Javascript syntax
2  let iterations=1000 // declares the number of iterations
3  const pi=3.14159265 // declares a constant
4
5  // declares a function
6  function calculatePi(iterations) {

```

```

17 // calls the function and store the result
18 let myComputationOfPi=calculatePi(iterations)
19 //console.log("Calculated value for pi: ", myComputationOfPi)
20 document.querySelector("#result").innerHTML=`Calculated value for pi: ${myComputationOfPi}`
21
22 // computes the error between calculated pi and real value of it
23 let error=pi-myComputationOfPi
24 //console.log(`Error for ${iterations} iterations: `, error)
25 document.querySelector("#error").innerHTML=`Error for ${iterations} iterations: ${error}`
26

```

3) JAVASCRIPT SYNTAX

- Variable names must be unique and can't:
 - start with a number or contain spaces or special characters (except _ and \$)
 - be a reserved keyword (e.g. var, let, const, function, etc.)

```
JS test.js > ...
1  /*
2  This program declares a variable ageUser and changes its value.
3  It also performs a "if" test to display a message in the console depending on the value of age.
4  */
5
6  let ageUser=0 // declares a variable ageUser with value 0
7
8  ageUser=52 // changes the value of the variable
9
10 console.log(`Because you are ${ageUser} years old, `)
11
12 if (ageUser<50) {
13 |   console.log('you are young!')
14 } else {
15 |   console.log('you are wise!') // not 'you are old!'
16 }
```

3) JAVASCRIPT SYNTAX

- Variable names must be unique and can't:
 - start with a number or contain spaces or special characters (except `_` and `$`)
 - be a reserved keyword (e.g. `var`, `let`, `const`, `function`, etc.)

```
JS test.js > ...
```

```
PS C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\01>
Because you are 52 years old,
you are wise!
PS C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\01>
6 let ageUser=0 // declares a variable ageUser with value 0
```

```
6 let ageUser=0 // declares a variable ageUser with value 0
7
8 ageUser=52 // changes the value of the variable
9
10 console.log(`Because you are ${ageUser} years old, `)
11
12 if (ageUser<50) {
13 |     console.log('you are young!')
14 } else {
15 |     console.log('you are wise!') // not 'you are old!'
16 }
```

3) JAVASCRIPT SYNTAX

- Multiple tests

```
JS test-multiple.js > ...
1  /*
2  This program declares a variable age
3  and performs a multiple "if" test.
4  */
5
6  let age=25;
7
8  if (age>=18) {
9      console.log('You are an adult.');
```

```
10 } else if (age>=13) {
11     console.log('You are a teenager.');
```

```
12 } else if (age>=3) {
13     console.log('You are a child.');
```

```
14 } else {
15     console.log('You are a baby.');
```

```
16 }
17
```


3) JAVASCRIPT SYNTAX

- Multiple tests

JS test-multiple.js > ...

```
1  /*
2  This program declares a variable age
3  and performs a multiple "if" test.
4  */
```

```
5
```

```
6
```

```
7  let age = 18;
8
9  if (age >= 18) {
10     console.log('You are an adult.');
```

```
11 } else if (age >= 13) {
```

```
12     console.log('You are a teenager.');
```

```
13 } else if (age >= 3) {
```

```
14     console.log('You are a child.');
```

```
15 } else {
```

```
16     console.log('You are a baby.');
```

```
17 }
```

3) JAVASCRIPT SYNTAX

- **Data type:** the type of value a variable can hold
- Some common data types in JavaScript:
 - String: 'Hello' , "World" - text
 - Number: 42 , 3.14 - numbers
 - Boolean: true , false - true or false
 - Array: [1, 2, 3] - a list of values

```
JS datatypes.js > ...
1  // This program demonstrates several different datatypes in JS
2
3  let firstName='Jérôme'
4  let age=52
5  let isTeacher=true
6  let hobbies=['football', 'walking in the forest', 'learning swedish']
7
8  console.log(firstName, ' is ', age, '. Is a teacher? ', isTeacher)
9  console.log('List of hobbies: ')
10 console.log(hobbies)
11
```

3) JAVASCRIPT SYNTAX

- **Data type:** the type of value a variable can hold
- Some common data types in JavaScript:
 - String: 'Hello' , "World" - text
 - Number: 42 , 3.14 - numbers
 - Boolean: true , false - true or false
 - Array: [1, 2, 3] - list of values

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node datatypes.js
Jérôme is 52 . Is a teacher? true
List of hobbies:
[ 'football', 'walking in the forest', 'learning swedish' ]
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>
```

```
1 // This program demonstrates several different datatypes in JS
2
3 let firstName='Jérôme'
4 let age=52
5 let isTeacher=true
6 let hobbies=['football', 'walking in the forest', 'learning swedish']
7
8 console.log(firstName, ' is ', age, '. Is a teacher? ', isTeacher)
9 console.log('List of hobbies: ')
10 console.log(hobbies)
11
```

3) JAVASCRIPT SYNTAX

- **Loops:** used when you need to repeat multiple times the same instructions

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node loops
```

```
i= 0
i= 1
i= 2
i= 3
i= 4
i= 5
i= 6
i= 7
i= 8
i= 9
k: 10
k: 9
k: 8
k: 7
k: 6
k: 5
k: 4
k: 3
k: 2
k: 1
k: 0
```

```
JS loops.js > ...
```

```
1 // Many loops in JS
2 let nbIterations=10;
3
4 ✓ for (let i=0; i<nbIterations; i++) {
5   |   console.log(`i= `, i)
6   | }
7
8   let k=nbIterations // we set the number of iterations
9 ✓ while(k>=0) {
10  |   console.log('k: '+k); // string concatenation
11  |   k--;
12  | }
```

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>
```

4) JAVASCRIPT FUNCTIONS

- **Functions:**
 - Reusable blocks of code
 - Can take inputs (parameters) and return a value

```
JS functions.js > ...
1  // Several functions in JS
2  function greetings(name) {
3      console.log(`Hej ${name}! Välkommen här.`)
4  }
5
6  function factorial(nb) {
7      if (nb<=1) {
8          return 1
9      } else {
10         return nb*factorial(nb-1)
11     }
12 }
13
14 let theName='Jérôme'
15 greetings(theName)
16 theName='Mira'
17 greetings(theName)
18
19 let theNumber=5
20 console.log(`${theNumber}!=`+factorial(theNumber))
21
```

4) JAVASCRIPT FUNCTIONS

- **Functions:**
 - Reusable blocks of code
 - Can take inputs (parameters) and return a value

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node functions.js
Hej Jérôme! Välkommen här.
Hej Mira! Välkommen här.
5!=120
```

```
JS functions.js > ...
1  // Several functions in JS
2  function greetings(name) {
3      console.log(`Hej ${name}! Välkommen här.`)
4  }
5
6  function factorial(nb) {
7      if (nb<=1) {
8          return 1
9      } else {
10         return nb*factorial(nb-1)
11     }
12 }
13
14 let theName='Jérôme'
15 greetings(theName)
16 theName='Mira'
17 greetings(theName)
18
19 let theNumber=5
20 console.log(`${theNumber}!=`+factorial(theNumber))
21
```

4) JAVASCRIPT FUNCTIONS

- **Arrow Functions** in Javascript:

4) JAVASCRIPT FUNCTIONS

- **Arrow Functions** in Javascript:

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node functions-arrow.js
Hej Jérôme! Välkommen här.
Hej Mira! Välkommen här.
5!=120
```

```
JS functions-arrow.js > ...
1  // Several functions using the arrow notation in JS
2  let greetings = (name) => {
3    console.log(`Hej ${name}! Välkommen här.`)
4  }
5
6  let factorial = (nb) => {
7    if (nb<=1) {
8      return 1
9    } else {
10     return nb*factorial(nb-1)
11   }
12 }
13
14 let theName='Jérôme'
15 greetings(theName)
16 theName='Mira'
17 greetings(theName)
18
19 let theNumber=5
20 console.log(`${theNumber}!=`+factorial(theNumber))
21
```


4) JAVASCRIPT FUNCTIONS

- **Arrow Functions** in Javascript:

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node functions-arrow2.js
12+9=21
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>
```

```
JS functions-arrow2.js > ...
1  // Several functions using the arrow notation in JS
2
3  // Instead of:
4
5  // function add(a, b) {
6  //   return a + b;
7  // }
8
9  // use:
10 const add = (a, b) => a + b;
11
12 console.log(`12+9=${add(12,9)}`)
13
```

5) JAVASCRIPT OBJECTS

- When you want to work with more **advanced datatypes**, you can use **objects**.

```
JS objects.js > ...
1  // Several objects in JS
2  let person1={
3      firstname: 'Jérôme',
4      lastname: 'Landré',
5      age: 52
6  }
7
8  let person2={
9      firstname: 'Mira',
10     lastname: 'Pop',
11     age: 25
12 }
13
14 console.log(`Hej ${person1.firstname}!`)
15
16 console.log(`Do you know ${person2.firstname}? She is ${person2['age']}.`)
17
```

5) JAVASCRIPT OBJECTS

- When you want to work with more **advanced datatypes**, you can use **objects**.

JS objects.js > ...

```
1 // Several objects in JS
2 let person1={
3     firstname: 'Jérôme',
4     lastname: 'Landré',
5     age: 52
```

7

```
8 let person2={
9     firstname: 'Mira',
10    lastname: 'Pop',
11    age: 25
12 }
```

13

```
14 console.log(`Hej ${person1.firstname}!`)
```

15

```
16 console.log(`Do you know ${person2.firstname}? She is ${person2['age']}.`)
```

17

```
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node objects.js
Hej Jérôme!
Do you know Mira? She is 25.
```

6) JAVASCRIPT ARRAYS

- You can also create arrays that contain many information in one place (one variable)

```
JS arrays.js > ...
1  ∨ // Several arrays in JS
2
3  // Attention, array indexes start from 0
4  let monthlyTemperaturesArray=[-1, 2, 4, 6, 5, 15, 21, 20, 18, 15, 8, 2]
5
6  ∨ for (let i=0; i<monthlyTemperaturesArray.length; i++) {
7    |   console.log(`Month ${i+1}: ${monthlyTemperaturesArray[i]} degree Celsius`)
8    | }
9
10 // Another way to do it: foreach
11 monthlyTemperaturesArray.forEach(temp => { console.log(`${temp} degree Celsius`) })
12
13 // An array of objects
14 let person1={ firstname: 'Jérôme', lastname: 'Landré', age: 52 }
15 let person2={ firstname: 'Mira', lastname: 'Pop', age: 25 }
16
17 let myPersonsArray=[ person1, person2]
18
19 myPersonsArray.forEach(p => { console.log(`Hej ${p.firstname}!`) })
20
```

6) JAVASCRIPT ARRAYS

- You can also create arrays that contain many information in one place (one variable)

```

JS arrays.js > ...
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>node arrays.js
Month 1: -1 degree Celsius
Month 2: 2 degree Celsius
Month 3: 4 degree Celsius
Month 4: 6 degree Celsius
Month 5: 5 degree Celsius
Month 6: 15 degree Celsius
Month 7: 21 degree Celsius
Month 8: 20 degree Celsius
Month 9: 18 degree Celsius
Month 10: 15 degree Celsius
Month 11: 8 degree Celsius
Month 12: 2 degree Celsius
-1 degree Celsius
2 degree Celsius
4 degree Celsius
6 degree Celsius
5 degree Celsius
15 degree Celsius
21 degree Celsius
20 degree Celsius
18 degree Celsius
15 degree Celsius
8 degree Celsius
2 degree Celsius
Hej Jérôme!
Hej Mira!
C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-javascript-5>
1 // Several arrays in JS
2
3 // Attention, array indexes start from 0
4 let monthlyTemperaturesArray=[-1, 2, 4, 6, 5, 15, 21, 20, 18, 15, 8, 2]
5
6 for (let i=0; i<monthlyTemperaturesArray.length; i++) {
7     console.log(`Month ${i+1}: ${monthlyTemperaturesArray[i]} degree Celsius`)
8 }
9
10 // Another way to do it: foreach
11 monthlyTemperaturesArray.forEach(temp => { console.log(`${temp} degree Celsius`) })
12
13 // An array of objects
14 let person1={ firstname: 'Jérôme', lastname: 'Landré', age: 52 }
15 let person2={ firstname: 'Mira', lastname: 'Pop', age: 25 }
16
17 let myPersonsArray=[ person1, person2]
18
19 myPersonsArray.forEach(p => { console.log(`Hej ${p.firstname}!`) })
20

```

7) INTERACTIONS JAVASCRIPT/DOM

- You can access the DOM using Javascript through the document object and functions

```
<> index-dom1.html > html > head > title
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <title>Accessing the DOM using Javascript</title>
6  </head>
7  <body>
8  |   <h1>Accessing the DOM</h1>
9  |   <p id="svenska" class="greetingMultiLanguages">Hej!</p>
10 |   <p id="english" class="greetingMultiLanguages">Hej!</p>
11 |   <p id="francais" class="greetingMultiLanguages">Hej!</p>
12 |
13 |   <script src="dom1.js"></script>
14 </body>
15 </html>
```

```
JS dom1.js > ...
```

```
1  document.getElementById("english").innerText="Hello!"
2  document.querySelector("#francais").innerText="Bonjour!"
3
4  document.querySelectorAll(".greetingMultiLanguages").forEach(elem => elem.style.color="violet")
```

Accessing the DOM

Hej!

Hello!

Bonjour!

7) INTERACTIONS JAVASCRIPT/DOM

- You can manipulate the DOM using Javascript functions:

Action	Function/Method
Select/Read	document.getElementById(id)
	document.querySelector(cssSelector)
	document.querySelectorAll(cssSelector)
	element.parentNode
	element.children
Add	document.createElement(tagName)
	element.appendChild(node)
	element.insertBefore(newNode, refNode)
	element.prepend(node)
Update	element.textContent = "text"
	element.innerHTML = "HTML"
	element.setAttribute(name, value)
	element.classList.add("class")
Delete	element.removeChild(child)
	element.remove()
Clone	element.cloneNode(deep)

7) INTERACTIONS JAVASCRIPT/DOM

- Javascript is an **event-based** language

<> index-dom2-events.html >  html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Accessing the DOM using Javascript</title>
6  </head>
7  <body>
8    <h1>Accessing the DOM</h1>
9    <p id="greetings">Hej!</p>
10
11    <button id="svenska">På svenska</button>
12    <button id="english">In English</button>
13    <button id="francais">En Francais</button>
14
15    <script src="dom2.js"></script>
16  </body>
17 </html>

```

JS dom2.js > ...

```

1  buttonSV=document.querySelector("#svenska")
2  buttonEN=document.querySelector("#english")
3  buttonFR=document.querySelector("#francais")
4  theText=document.querySelector("#greetings")
5
6  buttonSV.addEventListener('click', () => { theText.innerText="Hej!" })
7  buttonEN.addEventListener('click', () => { theText.innerText="Hello!" })
8  buttonFR.addEventListener('click', () => { theText.innerText="Bonjour!" })
9

```

Accessing the DOM

Bonjour!

På svenska

In English

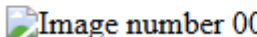
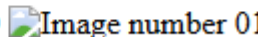
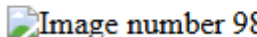
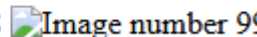
En Francais

7) INTERACTIONS JAVASCRIPT/DOM

- **DOMContentLoaded event**

- Runs when the HTML has been completely loaded and parsed
- Use this when your code needs to run after the HTML has loaded
- Not necessary when `<script>` is placed at the bottom of `<body>`

My image gallery:

  ...  

The page has been fully loaded and JS can now interact with it!...

```
<> index-pageloaded.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>The page is ready to be processed using JS</title>
6  </head>
7  <body>
8      <h1>My image gallery:</h1>
9      <div>
10         
11         
12         ...
13         
14         
15     </div>
16
17     <script src="pagefullyloaded.js"></script>
18 </body>
19 </html>
```

```
JS pagefullyloaded.js > ...
1  // The DOM is being built, it takes a long time to get all the images...
2
3  document.addEventListener('DOMContentLoaded', function() {
4      // all the images have been received
5      console.log(`The page has been fully loaded and JS can now interact with it!...`)
6  });
```

7) INTERACTIONS JAVASCRIPT/DOM

- Please **REMEMBER**:

JAVASCRIPT is an **asynchronous language**:
you don't know **when** the **code**
will be **run** by **the browser**!

8) BEST PRACTICES

- Use **meaningful names**: Choose descriptive names for variables, functions, and classes (e.g., calculateTotalPrice instead of calc)
- **Variable Declarations**
 - Prefer const over let to avoid unintended reassignments
 - Use let only when you need to reassign a variable
 - Avoid var due to its function scope and other issues
- **Pure functions**: Write functions that return the same output for the same input and have no side effects
- **Avoid global scope**: Declare variables and functions in the smallest possible scope

9) RESOURCES ABOUT JAVASCRIPT

- MDN Documentation <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- W3schools online tutorials <https://www.w3schools.com/js/default.asp>
- ECMAScript 16.0 official documentation <https://262.ecma-international.org/16.0/>
- Complete Javascript tutorial: <https://javascript.info/>

REMEMBER:

WEB DEVELOPMENT FUNDAMENTALS

WEB DEV IS FUN

ANY QUESTIONS?